

Detection of DoH Tunnels using Time-series Classification of Encrypted Traffic

Mohammadreza MontazeriShatoori, Logan Davidson, Gurdip Kaur, Arash Habibi Lashkari
Canadian Institute for Cybersecurity
University of New Brunswick
Fredericton, New Brunswick, Canada
m.montazeri,ldavids2,gurdip.kaur,a.habibi.l@unb.ca

Abstract—Computer networks have fallen easy prey to cyber attacks in the ever-evolving internet services. Domain Name System (DNS) has also not remained untouched with these cybercrime attempts. Encrypted HyperText Transfer Protocol (HTTP) traffic over Secure Socket Layer (SSL), alternatively called HTTPS, has succeeded to prevent DNS attacks to a great extent. To secure DNS traffic, the security community has introduced the concept of DNS over HTTPS (DoH) to improve user privacy and security by combating eavesdropping and DNS data manipulation on the way to prevent Man-in-the-Middle (MitM) attacks. This paper discusses one of the persistent security concerns, abuse of DNS protocol to create covert channels by tunneling data through DNS packets. We identify tunneling activities that utilize DNS communications over HTTPS by presenting a two-layered approach to detect and characterize DoH traffic using time-series classifiers.

Index Terms—DNS over HTTPS, DoH, encrypted traffic characterization, malicious DoH, machine learning, deep learning

I. INTRODUCTION

Domain Name System (DNS) is the internet backbone for providing a mapping between human-readable hostnames and computer understandable Internet Protocol (IP) addresses. DNS protocol is designed with a decentralized hierarchical approach. When a DNS client generates a DNS query asking for an IP address, the local DNS server responds after looking into its cache. It does not find the answer within cache memory, it forwards the DNS query to recursive DNS resolver which tracks down the DNS record with repetitive DNS queries to root name server, Top Level Domain (TLD) name server and authoritative name servers until it gets the target authoritative answer [1]. Fig. 1 shows the working of the domain name system with sequence number marked on the direction of flow.

DNS tunneling is a method of using DNS protocol as a mean of encapsulating data transmission between a client and a server. The data is coded within records of an otherwise normal DNS request and the server may or may not return with some data encoded in the corresponding DNS response.

There can be different motives for using DNS as a means of data transmission. Most importantly, many firewalls do not examine the contents and frequency of DNS packets, which makes it easier for this data transmission to go unnoticed. Secondly, since the packet will reach the modified nameservers over several hops through DNS recursion, the communication

is harder to detect and block using common methods such as IP blacklisting. Many malware has already abused this method to exfiltrate data or to create a communication channel with their command-and-control servers. This helps attackers to take some degrees of control over the infected devices and potentially using them to orchestrate attacks on other targets such as launching DDoS attacks on online services [25].

DNS tunnels can be detected with a variety of methods including domain name analysis and statistical analysis of DNS packets. Different approaches have been used to detect anomalous domain names, such as using statistical models. Frequency of DNS resolutions, length of the sub-domains, and the usage of TXT records are also among variables that can help detect DNS tunnels [12] [21]. Blocking DNS tunnels may be achieved through domain blacklisting, IP blacklisting, and dropping DNS packets that are thought to be malicious in nature.

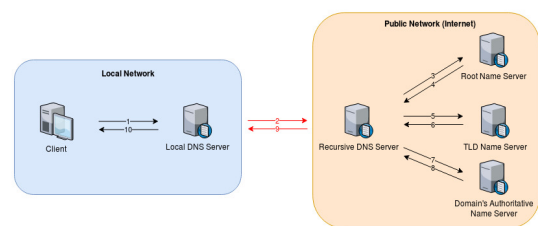


Fig. 1: Domain Name System [1]

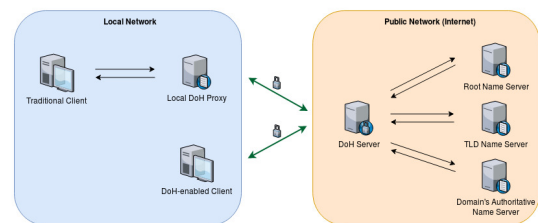


Fig. 2: DNS over HTTPS (DoH)

DoH has already been criticized by many security researchers for making DNS tunnels harder to detect and mitigate. First, Since the DoH wraps the DNS traffic in HTTPS,

the DNS traffic is imperceptible to the network infrastructure between the client (malware) and the DoH server. This effectively makes detection methods that rely on examining the DNS packets obsolete for the firewalls. Second, since HTTP/2 is the minimum version of HTTP that DoH standard recommends for using with DoH, Malware can utilize the HTTP/2 connection to send several DoH request, without creating a separate connection (or packet) for each request. The same also applies to the responses that DoH server is sending to the malware. Through this method, malware can hide the frequency of their DNS resolutions, further reducing the number of methods that can detect DNS tunnels [18].

The contributions of this paper are three-fold:

- Presenting a novel two-layered approach to classify DoH traffic from non-DoH traffic at layer 1 and characterize DoH traffic at layer 2.
- Generating a labeled dataset by capturing Benign-DoH, Malicious-DoH and non-DoH encrypted traffic in the network premises.
- Proposing a new feature-set based on time-series representation of traffic flows by introducing the concept of packet clumps and demonstrating the effectiveness of this feature set in encrypted traffic characterization.

Rest of this paper is organised as follows. Section II gives an overview of related work on traffic characterization. In section III, we provide the proposed methodology followed by detailed dataset in Section IV. Section V presents analyses and discussion of characterizing DoH traffic using a binary classifier. Finally, section VI concludes the paper and highlights research opportunities in this domain.

II. RELATED WORK

This section uncovers related work on encrypted and DNS tunneled traffic.

A. Encrypted Traffic Characterization

A thorough analysis is carried out to provide insights into lessons learned and challenges in the classification of encrypted traffic using deep learning [3] [4] [5]. Yang et al. [6] developed a payload-based classification method to identify Transport Layer Security (TLS) traffic by analyzing TLS connection handshake packets using the Bayesian neural network. Three parameters in the client *Hello* message viz cipher suites, compression methods, and TLS extensions were used to extract 392 features per flow.

Zhang et al. in [7] used HTTP/2 based website fingerprinting technique to classify encrypted TLS traffic based on local request and response sequence (LRRS) feature set. Similarly, Houser et al. [8] developed a DoT fingerprinting method to analyze DoT traffic generated by visiting websites. Siby et al. [9] provided a novel approach for website fingerprinting by analyzing DoH and DoT traffic. They used the patterns of encrypted DNS traffic to create fingerprints for each website.

Lotfollahi et al. [10] focused on feature extraction and classification to handle network traffic characterization and

identify end-user applications. Authors used Stacked AutoEncoder and Convolutional Neural Network for network classification. "ISCX VPN-nonVPN" dataset was used and the model obtained 98% accuracy for application identification and 93% accuracy for traffic characterization. Leroux et al. [11] also worked on ML models based on size and timing features to fingerprint VPN and ToR encrypted traffic.

Patsakis et al. [12] investigated the use of Domain Generation Algorithms (DGAs) to hide malicious DNS queries in a covert channel. They used Hodrick-Prezcott filter to classify traffic generated through several DGAs and studied the possibility of using DNSCurve and DNSCrypt by botnets to communicate with C&C server. Their work successfully constructed indicators of compromise (IoCs) even in the covert channel. They showcase the use of traffic analysis to provide a lightweight security mechanism and address future challenges dealing with masqueraded DNS queries.

Also, a group of researchers focused on developing the frameworks like MaMPF [13], Multi-Attribute Associated Fingerprints (MAAF) [14], and Software Defined Network (SDN) Home Gateway [15] to classify encrypted network traffic.

A Quick UDP Internet Connection (QUIC) protocol based CNN classifier is developed by Tong et al. [16] which used flow-based and packet-based features to identify some QUIC protocol based Google services with an accuracy of approximately 99%. However, the model suffered from high run-time of processing and classification due to use of flow-based features.

Wang et al. [17] proposed an end-to-end encrypted traffic classification method with 1D-CNN which integrated feature extraction, feature selection, and classifier into one end-to-end network.

B. DNS Tunneled Traffic Characterization

Qi et al. [19] proposed a bigram based DNS tunnel detection algorithm to define a scoring system for domain names to demonstrate the frequency of their bigrams in real domains. They used a dataset of real domains to create an online classifier to detect DNS tunnels by assigning a score to each DNS packet in the traffic. Their result showed improvement over earlier methods that used letter frequency and reduced the false positive rate significantly.

Buczak et al. [21] used traffic captured at each device and analyzed DNS tunneled data. Authors used random forest with features selected from the previous works on this area to show that the method is sufficiently effective even when the classifier had not seen the tunneling technique in the training set. They also determined the features that worked better with random forest classifier.

Ellens et al. [22] defined a flow-based DNS tunnel traffic detection method that grouped DNS packets with similar properties. Authors captured benign traffic in the network and used open-source tool to create tunneled traffic for web browsing, SSH communication and file transfer. Three categories of anomaly detectors are applied to detect anomalies in captured data without inspecting the packets deeply. Results showed

that flow-based detection is promising and capable of detecting all the tunnels used in the experiment irrespective of some false-positive results.

Do et al. [24] worked on tunneling traffic in mobile networks emphasizing on bypass procedures used by some mobile applications to avoid firewalls. Authors analyzed the shortcomings of payload and traffic analysis methods proposed earlier. They analyzed the legitimate and malicious traffic using k-means and One Class Super Vector Machine (OCSVM) to conclude that OCSVM with some specific settings can be used to detect DNS tunnels without using predetermined features used by previous researchers.

Packet-based method of classification is gaining importance since 2019 [34]. Moreover, Mouhammd and Tariq [32] presented a thorough review of available tools developed specifically for DNS tunneling and other tools that have other primary purposes but utilise DNS tunneling as a data exfiltration method. Details of experimentation with Iodine DNS tunnels have been given and it was shown how DNS tunneling can encapsulate other protocols such as SSH.

Liu et al. [33] discussed employing binary classifiers in recursive DNS servers for online DNS tunnel detection and prevention. Their proposed classifier uses 18 features constructed from the time interval between request and response, request packet size, sub-domain entropy, and record type of DNS communications. The input to the classifier is created by using a sliding window on the pairs of DNS request/response.

Nadler et al. [25] presented an architecture for detecting and blocking high-throughput and low-throughput data exfiltration over DNS. DNS queries are collected and several features are computed over a sliding window to use in isolation forest algorithm for detecting anomaly by calculating an anomaly score. The proposed method is evaluated using a real-world dataset from several public recursive DNS servers with the high volume of DNS traffic and is found to be a more robust way of detecting DNS tunnels, especially for the malware with low throughput traffics.

Although encrypted and DNS tunneled traffic characterization has received tremendous research focus as evident from review and survey papers in the past due to its implications on traffic shaping, flow control and congestion control, yet DoH traffic characterization is still an evolving concept. The research presented in this paper attempts to detect and characterize DoH traffic in an online environment. The main superiority of the proposed method is that it needs merely a small amount of traffic as input (less than 1 second) to competitively detect and characterize DoH traffic. To the best of our knowledge, this is the first work on DoH traffic detection and characterization.

III. PROPOSED METHODOLOGY

In this section, we provide the two-layer architecture used to classify (DoH and Non-DoH) in the first layer and characterize DoH traffic (benign DoH and malicious DoH) in the second layer with the detailed description of individual components,

feature set used, and classification technique (first contribution). Abnormal DoH detection methodology is described in Fig. 3 consisting of three modules: 1) Data pre-processing module, 2) Feature extraction module, and 3) Classification module.

A. Data capture and pre-processing module

The training dataset contains HTTPS traffic captured using various tools (detailed in Section IV). In the next step, the data pre-processing module identifies every flow captured from the encrypted network traffic by using tuple containing <source IP, destination IP, source port, destination port and protocol>. Dataset is labeled based on the destination IP address of flows because the protocol (TCP) and the destination port (443) remain the same for all the flows. DoH flows are also distinguished by tools used for generating them. Additionally, DoH flows created by simulating web browsing activity are marked as benign DoH while the flows captured using DoH tunnels are marked as malicious DoH. While DoH protocol may theoretically be abused by attackers in a variety of ways, we use "malicious" label for the DoH tunneling network traffic which can be used by malicious actors to create covert channels.

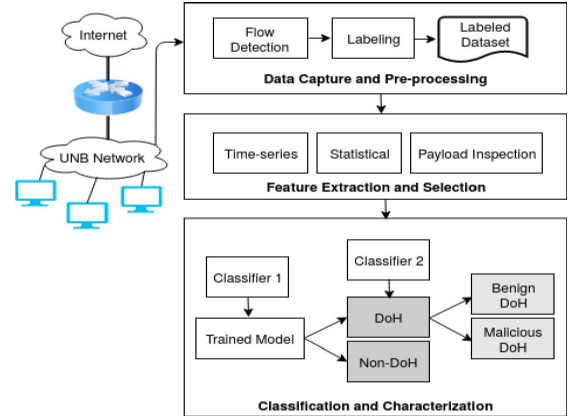


Fig. 3: DoH traffic classification and characterization architecture

B. Feature extraction and selection module

Feature extraction module is used to extract features from each flow. The prominent categories of features used for feature extraction as evident from review papers are time-series, statistical and payload inspection. We present a brief overview of these features here and provide the list of features used for this paper.

1) *Time-series features*: Time-series representation is used to model the network owing to the nature of traffic encrypted by TLS protocol in the form of a series of packets transmitted over the period. The nature of network traffic is a series of packets transmitting over time, so we can model a network flow using a time-series representation of captured traffic. Since the traffic used in this work is encrypted by TLS protocol, the payloads on the packets would not leak any useful information about the nature of underlying traffic. On the

contrary, we can use other traffic shape parameters such as the packet size, packet direction, and the time difference between packets, to infer some information about the underlying traffic [7]. We keep the packets that contain TLS application data and remove insignificant packets such as ACK packets with no payload and packets too small to carry data frames. The primary step in our method is to create packet clumps to reduce the dimensionality of data. We define a clump of packets as a sequence of one or more consecutive packets of a network flow (having the same source and destination) in the same direction to create a new and succinct representation of our data as shown in Fig. 4 (third contribution).

The rationale for this step is to combine these packets to find the application traffic scattered between several packets in the process of TLS segmentation and IP fragmentation. A threshold timeout value for clumps is also considered so that two packets with a greater time difference do not end up in the same packet clump. Algorithm 1 describes the clumping process. Each clump C is denoted by 5-tuple characteristics as:

$$C = \langle \text{size}, \text{pktCount}, \text{direction}, \text{duration}, \text{interarrival} \rangle$$

By using the clumping process, a sequence of clumps for any traffic flow is defined as

$$S = (C_1, \dots, C_n)$$

The size of these sequences (n) depends on the flow. We use a sliding window with a size of ℓ over this sequence of clumps to generate clump segments. Sequences that are smaller than the segment size are padded with empty clumps. Using ℓ as a hyper-parameter specifying the number of clumps in a segment, the final feature set F_ℓ extracted from a flow is represented as:

$$F_\ell = \{(C_i, \dots, C_{i+\ell}) \mid 1 \leq i < |S| - \ell\}$$

Finding the best value for ℓ is a trade-off between accuracy and response time. A smaller ℓ potentially limits the ability of the classifier to find meaningful patterns in the traffic. Thereby, it decreases the accuracy of results and time needed to capture traffic. In other words, it helps to detect and discard malicious traffic earlier.

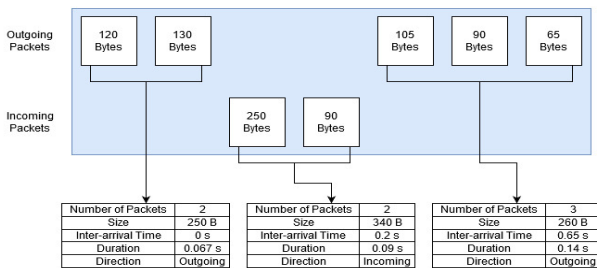


Fig. 4: Clumping process

Algorithm 1 Clumping Process

```

1: procedure CREATE_CLUMPS(packets)
2:   Initiate empty sequence S
3:   currentClump  $\leftarrow$  EMPTYCLUMP
4:   counter  $\leftarrow$  0
5:   while counter < packets.length do
6:     repeat
7:       Append packets[counter] to currentClump
8:       counter  $\leftarrow$  counter + 1
9:       timePassed  $\leftarrow$  packets[counter].time -
         packets[counter - 1].time
10:    until currentClump.direction  $\neq$ 
         packets[counter].direction or timePassed > timeout
11:    Append currentClump to S
12:    currentClump  $\leftarrow$  EMPTYCLUMP
13:  end while
14:  return S
15: end procedure

```

2) *Payload inspection features*: Since TLS traffic is encrypted, only the unencrypted handshake packets can be inspected. Following parameters in client *Hello* message can be used for application fingerprinting from these packets: list of available cipher suites, list of available compression methods, and list of available TLS extensions [6]. A fingerprint for each flow can be generated since each client has its own parameters for each flow. Although this fingerprinting method can be used to classify flows by clients yet it is not robust for detecting malicious tunneling traffic because it is limited to known client fingerprints only.

3) *Statistical features*: Statistical features are considered powerful for performing analyses. There are several related works on this domain which have used statistical features like duration of the flow, inter-arrival time etc. [28] [29]. Based on features used by these papers and our experiments, we have selected and developed 28 statistical features from captured DoH traffic as shown in Table I. To create the DoH flows (the combination of all request and response packets in one connection) and extract these features, we developed an open-source package by Python named DoHMeter which is publicly available in GitHub [23].

C. Classification module

Classification module is first used to train the classifier and create a trained model which is used to distinguish DoH traffic from non-DoH traffic at layer 1. After getting DoH traffic at layer 1, it is characterized as benign and malicious DoH traffic at layer 2.

1) *Statistical features classifier*: We used Random Forest (RF), Decision Tree (C4.5), Support Vector Machines (SVM), and Naive Bayes (NB) machine learning algorithms and deep neural network (DNN) and convolutional neural network (CNN) to classify statistical representation of flows. The results of these classifiers are compared to obtain the best classifier for our data. It is imperative to mention here that

TABLE I
STATISTICAL FEATURES

Parameter	Feature
F1	Number of flow bytes sent
F2	Rate of flow bytes sent
F3	Number of flow bytes received
F4	Rate of flow bytes received
F5-F12	Packet Length (F5: Mean, F6: Median, F7: Mode, F8: Variance, F9: Standard deviation, F10: Coefficient of variation, F11: Skew from median, F12: Skew from mode)
F13-20	Packet Time (F13: Mean, F14: Median, F15: Mode, F16: Variance, F17: Standard deviation, F18: Coefficient of variation, F19: Skew from median, F20: Skew from mode)
F21-F28	Request/response time difference (F21: Mean, F22: Median, F23: Mode, F24: Variance, F25: Standard deviation, F26: Coefficient of variation, F27: Skew from median, F28: Skew from mode)

we used DoHMeter to extract features from our dataset before applying statistical classifiers.

2) *Time-series Classifier*: For classifying time-series representations of our flows, we pre-processed the dataset to create clumps of data as explained in the previous subsection and then used deep learning classifiers to create a model capable of classifying sequences of clumps. Unlike the statistical features classifier that uses features calculated from the whole flow, time-series classifier can utilize a limited number of clumps to classify the flow. We used a deep neural network with 4 hidden layers (including an LSTM layer at the second hidden layer) to create our model. We used DoHlyzer which is a DoH traffic flow generator and analyzer for anomaly and attack detection and characterization. DoHlyzer is a script written in python that uses Scapy to read pcap files or sniff packets online. The tool is publicly available at [26].

IV. DATASET AND EXPERIMENTS

Our generated dataset, named *CIRA-CIC-DoHBrw-2020*, is publicly available at [27]. It consists of HTTPS traffic flows with two levels of distinct labels. We have DoH and Non-DoH HTTPS traffic at the first layer, and in the next layer, the DoH traffic is segregated as Benign DoH and Malicious DoH (second contribution). Although malicious traffic can be tunneling as well as non-tunneling but we generated only tunneled malicious traffic for this research. We used a variety of tools to simulate network activity to generate these flows as shown in Fig. 5. Firstly, normal web browsing activity that includes non-DoH HTTPS and benign DoH is simulated using Mozilla Firefox and Google Chrome web browsers. Secondly, malicious DoH is generated by a combination of tools used to create DoH tunnels as further detailed in this section. Traffic generated by all these tools is captured for pre-processing and training the classifiers.

To simulate web browsing network activity, we captured HTTPS traffic from web browsers when visiting a series of top 10k Alexa websites [20]. Since lazy loading of data is common in current web ecosystem, we refrained from using

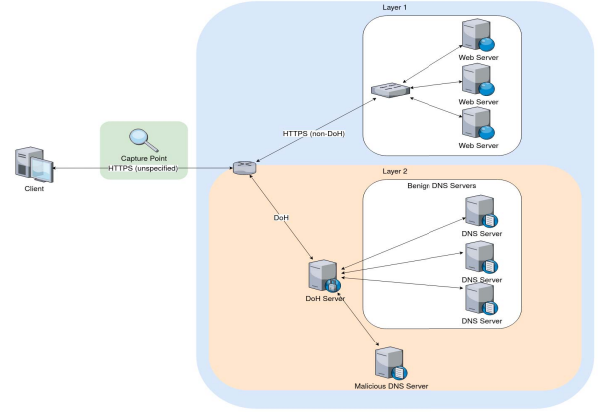


Fig. 5: Network diagram used to capture traffic

simple tools such as curl. The web browsers were configured to use various public DoH resolvers. Our web browsing dataset generation was implemented by using Python. We used the Selenium library, a front-end testing tool, in our Python script to communicate with the web browsers. To connect to Firefox, we also used GeckoDriver which is an intermediary between Firefox and tools that interacts with Firefox. Since we did not need to interact graphically with the web browser, we used a headless Firefox process that does not bring up the GUI. Furthermore, we set up Firefox in a way that used DoH instead of DNS, so that all the name resolution traffic happens to be DoH and could be captured. Similarly, for Google Chrome, we used chrome driver to communicate with the browser.

To create the malicious records of our dataset we deployed a network that can be used to simulate DoH tunneling scenarios. This is achieved by setting up a domain, setting our authoritative server as the nameserver for that domain, and using DoH requests/responses to carry data between client and server. Since the DoH protocol is backwards compatible with DNS authoritative nameservers, we used the existing DNS tunneling tools to setup our authoritative server. For the client, we used a DoH proxy to encapsulate every DNS requests received by the client into a DoH connection with a DoH Server and relay the response it received from the DoH server to the client as DNS records. Data is captured between the DoH proxy and DoH server. This set up allowed up to simulate DoH tunnels using various DNS tunneling tools that are used in the relevant studies [30]. To label the training dataset, these simulations were done separately in an isolated network so that any traffic captured can be associated with the correct label.

We developed a tool named *DoH Data Collector* to simulate different DoH tunneling scenarios and capture the resulting HTTPS traffic. In each instance of simulation, a new DoH tunnel is made over the underlying network according to different parameters used in a scenario such as:

- 1) DoH Server: Adguard, Cloudflare, Google, Quad9
- 2) DNS Tunneling Tool: Iodine, DNS2TCP, DNScat2

- 3) Tunneling Client and Server Configurations: Settings such as the delay between sending requests and DNS record types used.
- 4) Transmission Rate: Random value between 100 B/s to 1100 B/s
- 5) Duration

To generate enough data, the clients used in the simulation were run simultaneously on 10 servers, all connecting to a single C2 server posing as a DNS nameserver. A central controller based on another server controls the timing of simulations. This controller reads the scenarios from a configuration file with JSON format that defines each of the scenarios and uses SSH to access the clients and the server remotely to set up the simulation. Each simulation starts by running the C2 server application (such as Iodine) followed by running the client DoH proxies (which wraps DNS communications in a DoH connection) and the C2 client applications. During the simulation, a TCP connection with the specified transmission rate is tunneled through the C2 connection to generate the DoH traffic. We captured the traffic between the DoH proxy and DoH server using *tcpdump*.

TABLE II
DATASET DETAILS

Browser/Tool	DoH Server	Packets	Flows	Type
Google Chrome	AdGuard	5609K	105141	HTTPS (Non-DoH and Benign DoH)
	Cloudflare	6117K	132552	
	Google DNS	5878K	108680	
Mozilla Firefox	Quad9	10737K	199090	
	AdGuard	4943K	50485	
	Cloudflare	4299K	90260	
dns2tcp	Google DNS	6413K	138422	
	Quad9	4956K	92670	
	AdGuard	1281K	5459	Malicious DoH
DNSCat2	Cloudflare	3694K	6045	
	Google DNS	28711K	17423	
Iodine	Quad9	8750K	138588	
	AdGuard	1301K	5369	
	Cloudflare	12346K	9230	
	Google DNS	48069K	11915	
	Quad9	19309K	9108	
	AdGuard	3938K	11336	
	Cloudflare	5932K	14110	
	Google DNS	73459K	12192	
	Quad9	22668K	8975	

Working on all of the captured traffic, DoHMeter [23] was used to extract statistical and time-series features as mentioned in subsection III-B from the pcap files. Table II presents the details of captured packets and generated flows for different browsers and tools on four different DoH servers. A small number of flows ($n < 50$) in captured dataset contained NaN values for some of the features, mostly because the flow did not contain enough packets for those features to be calculated. These flows were deleted as part of the data pre-processing/cleaning procedure. The resulting dataset was divided to the training set (80% of the flows) and test set (the remaining 20%), to be used with all the classifiers discussed in this paper.

V. ANALYSES AND DISCUSSION

This section presents major findings and discussion of our research by analyzing the experimentation results in both layers of classification.

A. Classification by Statistical Features of the Flows

The results of classification of HTTPS traffic at layer 1 (DoH/non-DoH classes) using statistical features are presented in Table III. All the standard machine learning and deep neural network (DNN) and convolutional neural network (CNN) classifiers use statistical features calculated from the entire flow. Thus, it is important to investigate the duration of these flows for early detection of DoH traffic by an online classifier. All the classifiers share a mean delay of 20.393 seconds to detect whether a flow can be regarded as DoH. Apparently, random forest (RF) and C4.5 produced equivalent classification results with equal precision, recall and f-score value. It is followed by the support vector machine (SVM) and Naive Bayes (NB) at 0.877 and 0.84 precision, recall and f-score value respectively. We also evaluated deep neural network and two-dimensional (2D) CNN on the generated dataset with 0.97 and 0.98 precision respectively.

TABLE III
DoH TRAFFIC CLASSIFICATION BY ML/DL

Classifier	Precision	Recall	F-Score	Flow Duration (s)	
				Mean	Median
RF	0.993	0.993	0.993	20.393	1.397
C4.5	0.993	0.993	0.993		
SVM	0.877	0.877	0.877		
NB	0.84	0.834	0.833		
DNN	0.97	0.97	0.97		
2D CNN	0.98	0.98	0.98		

TABLE IV
DoH CHARACTERIZATION BY ML/DL

Classifier	Precision	Recall	F-Score	Flow Duration (s)	
				Mean	Median
RF	0.999	0.999	0.999	53.924	34.064
C4.5	0.999	0.999	0.999		
SVM	0.89	0.885	0.884		
NB	0.836	0.833	0.832		
DNN	0.98	0.98	0.98		
2D CNN	0.99	0.99	0.99		

The same feature set and classifiers are used at layer 2 to distinguish between malicious DoH (generated by tunneling activity) and benign DoH. Results presented in Table IV show the same pattern as layer 1 suggesting the advantage of utilizing the combination of statistical features and decision tree learning models. The increased flow duration from 20.393 seconds at layer 1 to 53.924 seconds at layer 2 indicates that DoH flows generally last longer compared to other non-DoH/HTTPS flows. All these classifiers give excellent results at both layers until we bring into consideration the fact the nature of DNS request-response pairs which is extremely short in duration to detect encrypted and DoH traffic. To overcome

this drawback, we formed clump sequence duration and results are explained in next sub-section.

B. Classification by Time-series Features of the Flows

We introduced *packet clumps* and *clump segments* to find patterns in a limited window of traffic which in turn reduces detection latency. We deployed long short-term memory (LSTM) a prominent recurrent neural network (RNN) architecture to create deep learning binary classifiers at both layers. Tables V and VI detail the results of using the proposed time-series feature set in combination with LSTM architecture. It gives an ostensible picture of how the value of all evaluation metrics keeps on escalating with the number of clumps (ℓ) formed at very short flow duration.

TABLE V
DoH TRAFFIC CLASSIFICATION BY LSTM

ℓ	Precision	Recall	F1-Score	Duration (s)	
				Mean	Median
1	0.877	0.876	0.876	0.081	0.000
2	0.950	0.949	0.949	0.167	0.002
3	0.953	0.951	0.951	0.262	0.010
4	0.983	0.983	0.983	0.366	0.020
5	0.987	0.987	0.987	0.468	0.030
6	0.993	0.993	0.993	0.574	0.047
7	0.996	0.996	0.996	0.675	0.060
8	0.996	0.996	0.996	0.775	0.077
9	0.998	0.998	0.998	0.871	0.091
10	0.998	0.998	0.998	0.964	0.105

TABLE VI
DoH CHARACTERIZATION BY LSTM

ℓ	Precision	Recall	F1-Score	Duration (s)	
				Mean	Median
1	0.825	0.792	0.782	0.164	0.002
2	0.985	0.984	0.985	0.329	0.022
3	0.991	0.991	0.991	0.502	0.05
4	0.995	0.995	0.995	0.685	0.094
5	0.997	0.997	0.997	0.872	0.142
6	0.998	0.998	0.998	1.063	0.203
7	0.997	0.997	0.997	1.26	0.258
8	0.999	0.999	0.999	1.45	0.313
9	0.999	0.999	0.999	1.63	0.373
10	0.999	0.999	0.999	1.803	0.44

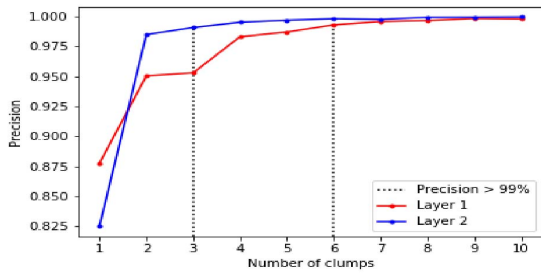
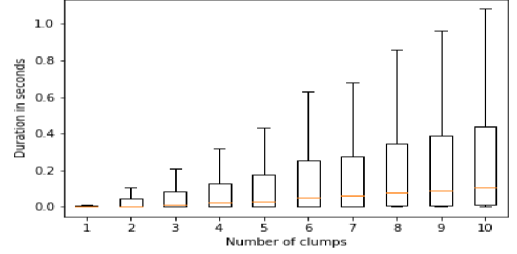
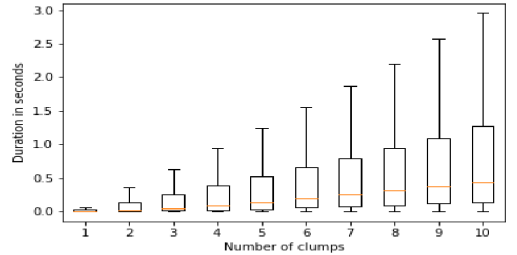


Fig. 6: Comparison of precision values at both layers

We consider the plot in Fig. 6 to actuate the threshold value for the number of clumps after which DoH flows can be classified at layer 1 and characterized at layer 2. It is observed that after six clumps in layer 1 and three clumps at layer 2, the precision hikes beyond 0.99 offering a precision comparable with the most accurate statistical classifiers.



(a) Layer 1



(b) Layer 2

Fig. 7: Distribution of clump sequence duration

To further analyze the clumping process used in our research, we mapped out a diagram between the number of clumps and distribution of clump segment duration in both layers in Fig. 7. With regards to the threshold value $\ell_1 \geq 6$ for the first layer, we can derive that a DoH connection can be detected roughly under 0.8 seconds since all the segments with six clumps created from the layer 1 traffic have values less than 0.8 (excluding the statistically insignificant outliers). The same conclusion can be made for layer 2 with a threshold value of $\ell_2 \geq 3$, placing the delay of our proposed time-series classifier at less than 1 second for DoH tunnel detection.

VI. CONCLUSION AND FUTURE SCOPE

We presented the exploitation of DNS protocol to create covert channels by tunneling data through DoH connections. DoH was successfully deployed through web browsers and DNS tunneling tools to generate and capture benign and malicious DoH flows along with encrypted traffic. We used statistical features to detect DoH connections and malicious DoH activity (DoH tunneling) in a two-layered binary classification approach where the first layer distinguishes DoH traffic from non-DoH traffic and the second layer characterizes malicious and benign DoH flows.

We demonstrated that malicious DoH traffic can be accurately detected by using the proposed two-layer binary classification architecture. Moreover, we dealt with latency as

a significant constraint when analyzing DoH request-response flows. We showed that clumping process facilitated early analysis of traffic allowing to create time-series classifiers capable of timely detection of malicious traffic while retaining the accuracy of statistical classifiers.

DoH was designed and implemented fairly recently especially in comparison to DNS. Therefore, it leverages a wide scope for research in security and privacy issues that may arise from its widespread adoption. In the future work, our objective is to focus on security vulnerabilities arising from other methods of DoH deployment in the network, especially local DoH proxies deployed in the network providing DoH service (DoHS).

ACKNOWLEDGEMENT

This research is supported by the Canadian Internet Registration Authority (CIRA)'s community investment program.

REFERENCES

- [1] Domain Name System - Request For Comments 1034, <https://tools.ietf.org/html/rfc1034>, Last accessed December 03, 2019
- [2] DNS Queries over HTTPS (DoH) - Request For Comments 8484, <https://tools.ietf.org/html/rfc8484>, Last accessed December 03, 2019
- [3] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, J. Aguilar, Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey. *IEEE Communications Surveys and Tutorials*, 21(2), pp. 1988-2014, 2019
- [4] G. Aceto, D. Ciunzo, A. Montieri, A. Pescapé, Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. *IEEE Transactions on Network and Service Management*, 16(2), pp. 445-458, 2019
- [5] S. Rezaei, X. Liu, Deep Learning for Encrypted Traffic Classification: An Overview. *IEEE Communications Magazine*, 57(5), pp. 76-81, 2019
- [6] J. Yang, J. Naranjaya, H. Lim, Bayesian Neural Network Based Encrypted Traffic Classification using Initial Handshake Packets. *Proceedings - 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume*, pp. 19-20, 2019
- [7] Z. Zhang, C. Kang, G. Xiong, Z. Li, Deep Forest with LRRS Feature for Fine-grained Website Fingerprinting with Encrypted SSL/TLS. *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pp. 851-860, 2019
- [8] H. Rebekah, L. Zhou, C. Chase, W. Haining, An Investigation on Information Leakage of DNS over TLS, In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies*, 2019
- [9] S. Sandra, J. Marc, D. Claudia, V. Narseo, T. Carmela, Encrypted DNS - Privacy? A Traffic Analysis Perspective. *Cryptography and Security*, Cornell University, 2019. Available at <https://arxiv.org/abs/1906.09682>
- [10] M. Lotfollahi, M. Siavoshani, Z. Shirali, M. Saberian, Deep packet: a novel approach for encrypted traffic classification using deep learning. *Soft Computing*, pp. 1-14, 2019
- [11] S. Leroux, S. Bohez, J. Maenhaut, N. Meheus, P. Simoens, B. Dhoedt, Fingerprinting encrypted network traffic types using machine learning. *IEEE/IFIP Network Operations and Management Symposium: Cognitive Management in a Cyber World*, 1-5, 2018
- [12] C. Patsakis, F. Casino, V. Katos, Encrypted and covert DNS queries for botnets: Challenges and countermeasures, *Computers & Security*, 88, 2020
- [13] C. Liu, Z. Cao, G. Xiong, G. Gou, M. Yiu, L. He, MaMPF: Encrypted Traffic Classification Based on Multi-Attribute Markov Probability Fingerprints. *2018 IEEE/ACM 26th International Symposium on Quality of Service*, pp. 1-10, 2018
- [14] Y. Chen, T. Zang, Y. Zhang, Y. Zhou, Y. Wang, Rethinking Encrypted Traffic Classification: A Multi-Attribute Associated Fingerprint Approach. *2019 IEEE 27th International Conference on Network Protocols*, pp. 1-11, 2019
- [15] P. Wang, F. Ye, X. Chen, Y. Qian, Datanet: Deep Learning Based Encrypted Network Traffic Classification in SDN Home Gateway. *IEEE Access*, 6, pp. 55380-55391, 2018
- [16] V. Tong, A. Tran, S. Souihi, A. Mellouk, A Novel QUIC Traffic Classifier Based on Convolutional Neural Networks. *2018 IEEE Global Communications Conference*, pp. 1-6, 2018
- [17] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks. *2017 IEEE International Conference on Intelligence and Security Informatics*, pp. 43-48, 2017
- [18] D. Hjelm, A New Needle and Haystack: Detecting DNS over HTTPS Usage. Available at <https://www.sans.org/reading-room/whitepapers/dns/needle-haystack-detecting-dns-https-usage-39160>, Last accessed April 27, 2020
- [19] C. Qi, X. Chen, C. Xu, J. Shia, P. Liu, A Bigram Based Real Time DNS Tunnel Detection Approach. *Information Technology and Quantitative Management*, *Procedia Computer Science*, Vol. 17, pp. 852-860, 2013
- [20] Alexa, <https://www.alexa.com/>, Last accessed December 10, 2019
- [21] A. Buczak, P. Hanke, G. Cancro, M. Toma, L. Watkins, J. Chavis, Detection of Tunnels in PCAP Data by Random Forests, *Proceedings of the 11th Annual Cyber and Information Security Research Conference*, pp. 1-4, 2016
- [22] W. Ellens, P. Żuranowski, A. Sperotto, H. Schotanus, M. Mandjes, E. Meeuwissen, Flow-Based Detection of DNS Tunnels, In: Doyen G., Waldburger M., Čeleda P., Sperotto A., Stiller B. (eds) *Emerging Management Mechanisms for the Future Internet*, *Lecture Notes in Computer Science*, Vol. 7943, Springer, Berlin, Heidelberg, 2013
- [23] DoHlyzer, <https://github.com/ahlashkari/DOHlyzer/tree/master/DoHMeter>, 2019
- [24] V. Do, P. Engelstad, B. Feng, T. Do, Detection of DNS tunneling in mobile networks using machine learning, In: Kim K., Joukov N. (eds) *Information Science and Applications 2017*, *Lecture Notes in Electrical Engineering*, Vol. 424, Springer, Singapore, 2017
- [25] A. Nadler, A. Aminov, A. Shabtai, Detection of malicious and low throughput data exfiltration over the DNS protocol, *Computers & Security*, Vol. 80, pp. 36-53, 2019
- [26] DoHlyzer, <https://github.com/ahlashkari/DOHlyzer>, 2019
- [27] CIRA-CIC-DoHBrw-2020, <https://www.unb.ca/cic/datasets/dohbrw-2020.html>, 2020
- [28] L. Arash, D. Gerard, M. Mohammad, G. Ali, Characterization of Tor Traffic Using Time-Based Features. In the proceeding of the 3rd International Conference on Information System Security and Privacy, SCITEPRESS, Portugal, pp. 253-262, 2017
- [29] D. Gerard, L. Arash, M. Mohammad, G. Ali, Characterization of Encrypted and VPN Traffic Using Time-Related Features, In *Proceedings of the 2nd International Conference on Information Systems Security and Privacy*, pp. 407-414, Italy, 2016
- [30] A. Merlo, G. Papaleo, S. Veneziano, M. Aiello, A Comparative Performance Evaluation of DNS Tunneling Tools, In: Herrero Á., Corchado E. (eds) *Computational Intelligence in Security for Information Systems*, *Lecture Notes in Computer Science*, Vol. 6694, Springer, Berlin, Heidelberg, 2011
- [31] R. Hasibi, M. Shokri, M. Dehghan, Augmentation Scheme for Dealing with Imbalanced Network Traffic Classification Using Deep Learning, 2019. Available at <http://arxiv.org/abs/1901.00204>
- [32] Al-kasassbeh, Mouhammd, and Tariq Khairallah. , Winning tactics with DNS tunnelling, *Network Security*, Vol 12, 2019
- [33] J. Liu, S. Li, Y. Zhang, J. Xiao, P. Chang and C. Peng, Detecting DNS tunnel through binary-classification based on behavior features. *2017 IEEE Trustcom/BigDataSE/ICSS*. IEEE, 2017
- [34] C. Liu, L. Dai, W. Cui and T. Lin, A Byte-level CNN Method to Detect DNS Tunnels, *2019 IEEE 38th International Performance Computing and Communications Conference (IPCCC)*, IEEE, 2019